

Design and Analysis of Algorithm

Dynamic Programming

Algorithm design techniques

Iterative (brute-force) algorithms

✚ Example: insertion sort, bubble sort, selection sort

Algorithms that use efficient data structures

✚ Example: heap sort

Divide-and-conquer algorithms

✚ Example: binary search, merge sort, quick sort

Greedy algorithms

✚ Example: minimum spanning tree, dijkstra

Graph algorithms

✚ Example: bfs, dfs

Dynamic programming

✚ Example: bellman-ford, longest common subsequence, longest increasing subsequence, 0/1 knapsack problem, matrix chain multiplication

History

Dynamic programming

- ❏ Invented in the 1957 by *Richard Bellman* as a general method for optimizing multistage decision processes
- ❏ The term “programming” refers to a *tabular method*.
- ❏ Often used for *optimization* problems.

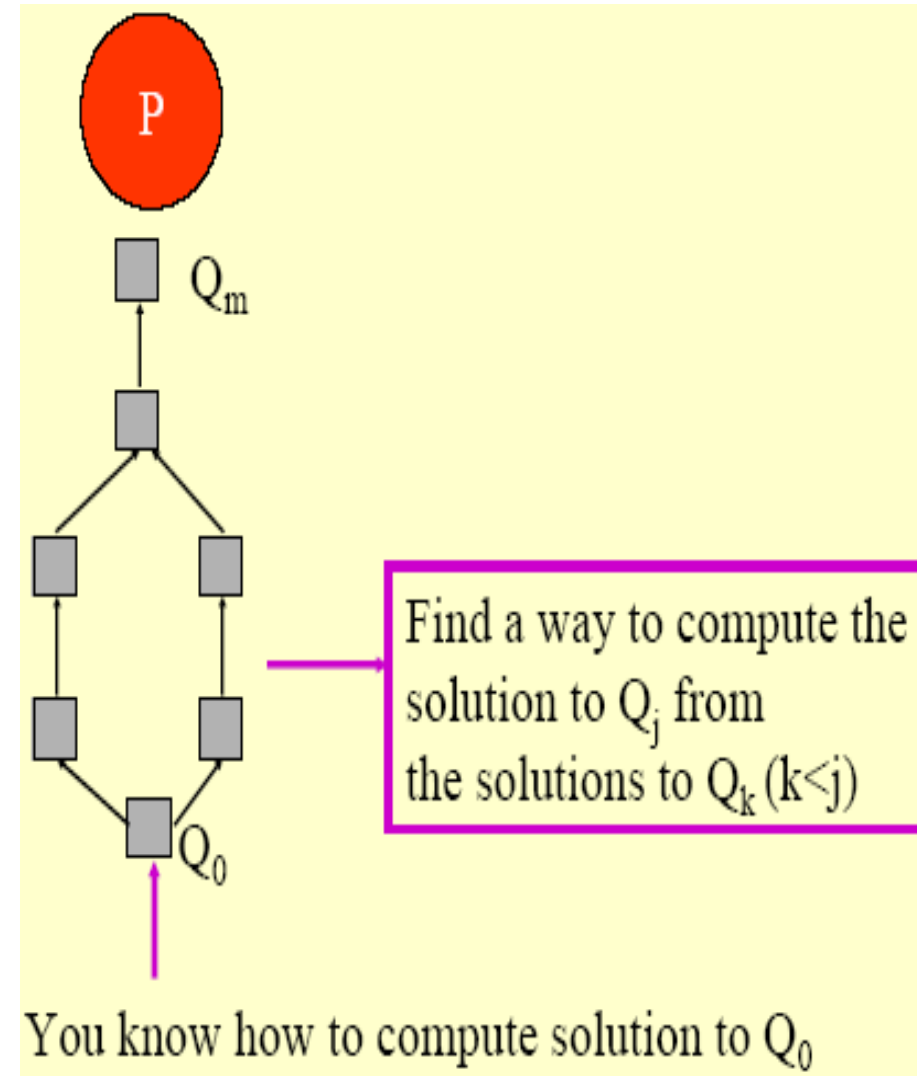
Dynamic Programming

- # Solves problems by **combining** the solutions to subproblems that contain common sub-sub-problems.
- # Difference between DP and Divide-and-Conquer
 - ✚ Using ***Divide and Conquer*** to solve these problems is **inefficient** as the same common sub-sub-problems have to be solved **many times**.
 - ✚ DP will solve each of them **once** and their **answers are stored in a table** for future reference.

Intuitive Explanation

Given a problem **P**, obtain a sequence of problems $Q_0, Q_1, \dots, Q_m$, where:

- ▣ You have a solution to Q_0
- ▣ The solution to a problem Q_j , $j > 0$, can be obtained from solutions to problems $Q_0 \dots Q_k$, $k < j$, that appear earlier in the “sequence”.



Elements of Dynamic Programming

DP is used to solve problems with the following characteristics:

✚ **Optimal sub-structure** (Principle of Optimality)

◆ an optimal solution to the problem contains within it optimal solutions to sub-problems.

✚ **Overlapping subproblems**

◆ there exist some places where we solve the same subproblem more than once

Steps to Designing a Dynamic Programming Algorithm

1. Characterize *optimal sub-structure*
2. ***Recursively*** define the value of an optimal solution
3. Compute the value *bottom up*
4. (if needed) ***Construct*** an optimal solution

1.Optimal Sub-Structure

An optimal solution to the problem contains optimal solutions to sub-problems

▣ Solution to a problem:

- ◆ Making a **choice** out of a number of **possibilities** (look what possible choices there can be)
- ◆ **Solving** one or more **sub-problems** that are the **result of a choice** (characterize the space of sub-problems)

▣ Show that solutions to sub-problems must themselves be optimal for the whole solution to be optimal.

2. Recursive Solution

Write a recursive solution for the value of an optimal solution.

$$M_{opt} = \underset{\text{over all } k \text{ choices}}{\text{Optimal}} \left[\begin{array}{l} \text{Combination of } M_{opt} \text{ for all subproblems resulting from choice } k \\ + \text{ The cost associated with making the choice } k \end{array} \right]$$

Show that the number of different instances of sub-problems is bounded by a polynomial.

3. Bottom Up

- # **Compute** the value of an optimal solution in a bottom-up fashion, so that you always have the necessary **sub-results pre-computed** (or use memoization)
- # Check if it is possible to **reduce** the **space** requirements, by “**forgetting**” solutions to sub-problems that will not be used any more

4. Construct Optimal Solution

#Construct an optimal solution from **computed information** (which records a sequence of choices made that lead to an optimal solution)

Memoization

- # ***Memoization* is one way to deal with overlapping subproblems**
 - ▣ After computing the solution to a subproblem, store it in a table
 - ▣ Subsequent calls just do a table lookup
- # **Can modify recursive algorithm to use memoization**

Memoization

Memoization:

- ❏ Solve the problem in a ***top-down*** fashion, but record the solutions to subproblems in a table.

Pros and cons:

- ❏ Recursion is usually slower than loops and uses stack space
- ❏ Easier to understand
- ❏ If not all subproblems need to be solved, you are sure that only the necessary ones are solved